

Segment Tree

Hall No : 2303a52189

Name : K.Srija

1. Range GCD Query with Point Updates

```
import java.util.*;
public class Main {
    static int[] tree, arr;
    static int gcd(int a, int b) {
        while (b != 0) {
            int t = a % b;
            a = b;
            b = t;
        }
        return a;
    }
    static void build(int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid);
        build(2 * node + 1, mid + 1, r);
        tree[node] = gcd(tree[2 * node], tree[2 * node + 1]);
    }

    static void update(int node, int l, int r, int idx, int val) {
        if (l == r) {
            tree[node] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (idx <= mid)
            update(2 * node, l, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, r, idx, val);
        tree[node] = gcd(tree[2 * node], tree[2 * node + 1]);
    }

    static int query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql)
            return 0;
        if (ql <= l && r <= qr)
```

```

        return tree[node];
    int mid = (l + r) / 2;
    return gcd(
        query(2 * node, l, mid, ql, qr),
        query(2 * node + 1, mid + 1, r, ql, qr)
    );
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    arr = new int[n];
    tree = new int[4 * n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();
    build(1, 0, n - 1);
    int q = sc.nextInt();
    while (q-- > 0) {
        int type = sc.nextInt();
        if (type == 1) {
            int index = sc.nextInt();
            int value = sc.nextInt();
            update(1, 0, n - 1, index, value);
        } else {
            int l = sc.nextInt();
            int r = sc.nextInt();
            System.out.println(query(1, 0, n - 1, l, r));
        }
    }
}
}
}
}

```

2. Range Product Query with Point Updates

```

import java.util.*;
public class Main {
    static long[] tree, arr;
    static final long MOD = 1000000007L;
    static void build(int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l] % MOD;
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid);
        build(2 * node + 1, mid + 1, r);
        tree[node] = (tree[2 * node] * tree[2 * node + 1]) % MOD;
    }
}

```

```

static void update(int node, int l, int r, int idx, long val) {
    if (l == r) {
        tree[node] = val % MOD;
        return;
    }
    int mid = (l + r) / 2;
    if (idx <= mid)
        update(2 * node, l, mid, idx, val);
    else
        update(2 * node + 1, mid + 1, r, idx, val);
    tree[node] = (tree[2 * node] * tree[2 * node + 1]) % MOD;
}

static long query(int node, int l, int r, int ql, int qr) {
    if (qr < l || r < ql)
        return 1;
    if (ql <= l && r <= qr)
        return tree[node];
    int mid = (l + r) / 2;
    return (query(2 * node, l, mid, ql, qr)
            * query(2 * node + 1, mid + 1, r, ql, qr)) % MOD;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    arr = new long[n];
    tree = new long[4 * n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextLong();
    build(1, 0, n - 1);
    int q = sc.nextInt();
    while (q-- > 0) {
        int type = sc.nextInt();
        if (type == 1) {
            int index = sc.nextInt();
            long value = sc.nextLong();
            update(1, 0, n - 1, index, value);
        } else {
            int l = sc.nextInt();
            int r = sc.nextInt();
            System.out.println(query(1, 0, n - 1, l, r));
        }
    }
}

```

3. Range Bitwise OR Query with Point Updates

```

import java.util.*;
public class Main {
    static int[] tree, arr;
    static void build(int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid);
        build(2 * node + 1, mid + 1, r);
        tree[node] = tree[2 * node] | tree[2 * node + 1];
    }
    static void update(int node, int l, int r, int idx, int val) {
        if (l == r) {
            tree[node] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (idx <= mid)
            update(2 * node, l, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, r, idx, val);
        tree[node] = tree[2 * node] | tree[2 * node + 1];
    }
    static int query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql)
            return 0;
        if (ql <= l && r <= qr)
            return tree[node];
        int mid = (l + r) / 2;
        return query(2 * node, l, mid, ql, qr)
            | query(2 * node + 1, mid + 1, r, ql, qr);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        arr = new int[n];
        tree = new int[4 * n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        build(1, 0, n - 1);
        int q = sc.nextInt();
        while (q-- > 0) {
            int type = sc.nextInt();
            if (type == 1) {
                int index = sc.nextInt();

```

```

        int value = sc.nextInt();
        update(1, 0, n - 1, index, value);
    } else {
        int l = sc.nextInt();
        int r = sc.nextInt();
        System.out.println(query(1, 0, n - 1, l, r));
    }
}
}
}
}

```

4. K-th One in a Binary Array

```

import java.util.*;

public class Main {
    static int[] tree, arr;
    static void build(int node, int l, int r) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid);
        build(2 * node + 1, mid + 1, r);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    static void update(int node, int l, int r, int idx, int val) {
        if (l == r) {
            tree[node] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (idx <= mid)
            update(2 * node, l, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, r, idx, val);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    static int kthOne(int node, int l, int r, int k) {
        if (l == r)
            return l;
        int mid = (l + r) / 2;
        if (tree[2 * node] >= k)
            return kthOne(2 * node, l, mid, k);
        else
            return kthOne(2 * node + 1, mid + 1, r,

```

```

        k = tree[2 * node];
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        arr = new int[n];
        tree = new int[4 * n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        build(1, 0, n - 1);
        int q = sc.nextInt();
        while (q-- > 0) {
            int type = sc.nextInt();
            if (type == 1) {
                int index = sc.nextInt();
                int value = sc.nextInt();
                update(1, 0, n - 1, index, value);
            } else {
                int k = sc.nextInt();
                if (tree[1] < k)
                    System.out.println(-1);
                else
                    System.out.println(kthOne(1, 0, n - 1, k));
            }
        }
    }
}

```